```
  digitalWrite(ledPin, ledState);
  delay(20);
}
```

This example uses two variables, `state` and `lastState`, to keep track of the state of the input pin at any given time. A third variable, `ledState`, is used to toggle the state of the LED. In this way, when the button is pressed the first time, the LED will turn on, press the button a second time and the LED will turn off, press the button a third time, the LED will come back on, and so on. Let's unpack the code a bit before moving to the next example, beginning with the first line in the `loop()` function.

```
state = digitalRead(switchPin);
```

This line reads the input from the pin connected to our push button and assigns its value to the variable state.

```
if (state != lastState) {
```

Our first of three conditional statements, this line is what this whole sketch hinges on. It checks to see if the current state is different from the state the last time it checked. By using the `!=` operator, this line in English is like saying, "If the current state is not equal to the last state then execute the following lines." Notice, it doesn't actually matter at this point what the exact state is of the pin, only that it's different.

```
if (state == HIGH) {
```

This line will check to see what the state actually is so that the state of the LED can be changed accordingly in the next two lines.

```
if (ledState == HIGH) ledState = LOW;
else ledState = HIGH;
```

This `if…else` statement will either change the state of the LED to `LOW` if it was `HIGH` last time or `HIGH` if it was already `LOW`. This statement doesn't actually turn on or off the LED, but we will get to that in a moment.

```
lastState = state;
```

This line is the counterpart to the statement, `if (state != lastState)`, and is responsible for changing the current state of the input pin to the old state before we assign a new value to the variable state.

```
digitalWrite(ledPin, ledState);
delay(20);
```

These last two lines will set the `ledPin` to whatever `ledState` is equal to, either turning on or off the LED. The very short 20 millisecond delay is there just to slow things down enough so that the sketch reads one press of the push button as opposed to multiples.

Counting

Our last example toggled the LED on and off whenever the button was pressed. But what if you want the LED to turn on only once every five button presses? To do that we need to set up a counter that will count the number of presses. Listing 5-3 shows one way that this could be done.